Time Series Transformations and Decompositions

Set up

We always load the following packages.

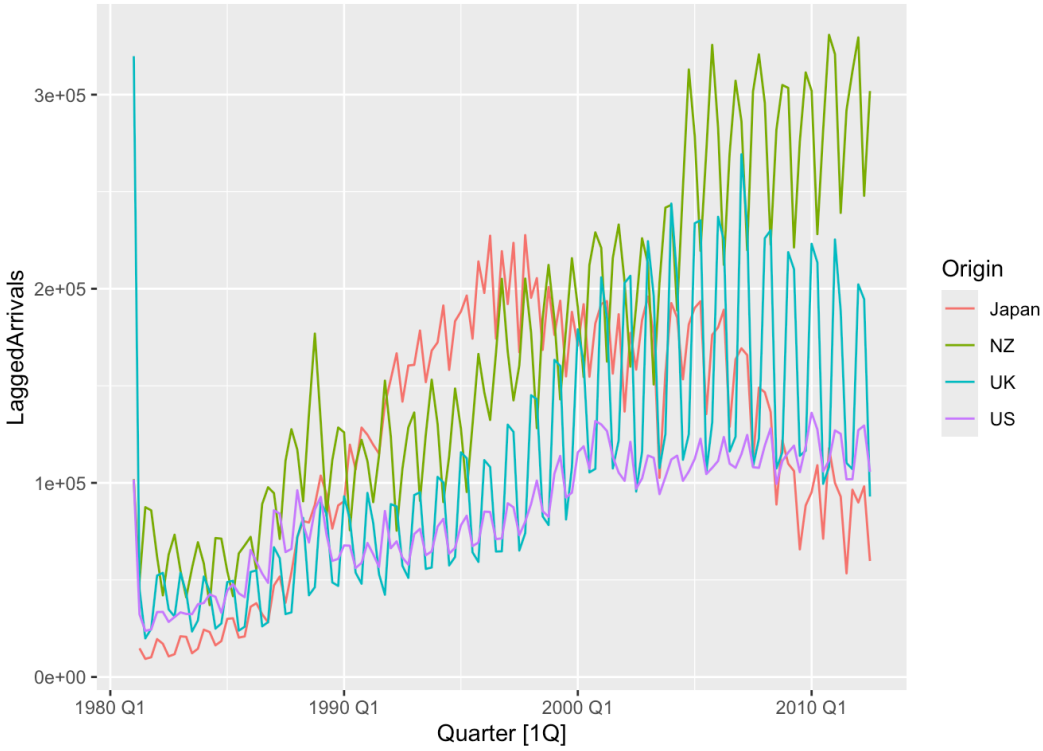
```
library(fpp3)
library(tidyverse)
```

1. Lags and differencing

Let's consider the `aus_arrivals` dataset and try to plot lagged arrival values.

```
aus_arrivals |>
  mutate(LaggedArrivals = lag(Arrivals)) |>
  autoplot(LaggedArrivals)
```

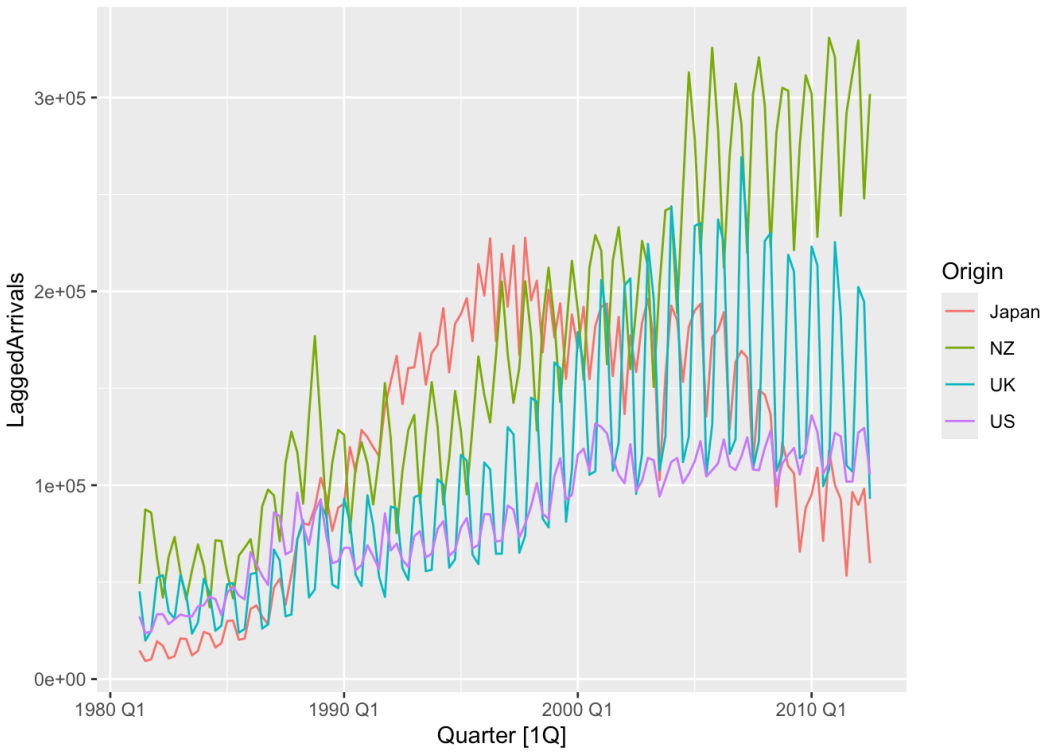
```
Warning: Removed 1 row containing missing values or values
outside the scale range
(`geom_line()`).
```



Something seems a bit odd about this plot, especially the UK value. What is happening is that the lag function has treated the entire column as a single time series even though it contains 4 separate time series. To avoid this, we need to do a `group_by()` before applying mutate

```
aus_arrivals |>
  group_by(Origin) |>
  mutate(LaggedArrivals = lag(Arrivals)) |>
  ungroup() |>
  autoplot(LaggedArrivals)
```

```
Warning: Removed 4 rows containing missing values or values
outside the scale range
(`geom_line()`).
```



2. Moving averages

Let's focus on the arrivals from Japan time series and compute moving averages.

```
library(slider)

aus_jap_arrivals <- aus_arrivals |>
  filter(Origin == "Japan") |>
```

```
  mutate(MA2 = slide_dbl(Arrivals, mean, .before = 1, .after = (
         MA2_alt = (Arrivals + lag(Arrivals)) / 2)
aus_jap_arrivals
```

```
# A tsibble: 127 x 5 [1Q]
# Key:       Origin [1]
   Quarter Origin Arrivals    MA2 MA2_alt
     <qtr> <chr>     <int>  <dbl>   <dbl>
 1 1981 Q1 Japan     14763 14763      NA
 2 1981 Q2 Japan      9321 12042   12042
 3 1981 Q3 Japan     10166  9744.   9744.
 4 1981 Q4 Japan     19509 14838.  14838.
 5 1982 Q1 Japan     17117 18313   18313
 6 1982 Q2 Japan     10617 13867   13867
 7 1982 Q3 Japan     11737 11177   11177
 8 1982 Q4 Japan     20961 16349   16349
 9 1983 Q1 Japan     20671 20816   20816
10 1983 Q2 Japan     12235 16453   16453
# ℹ 117 more rows
```

Manually computing the moving average shows that the formula is correct. We now compute moving averages with a few different window sizes to see the effect of varying the window size.

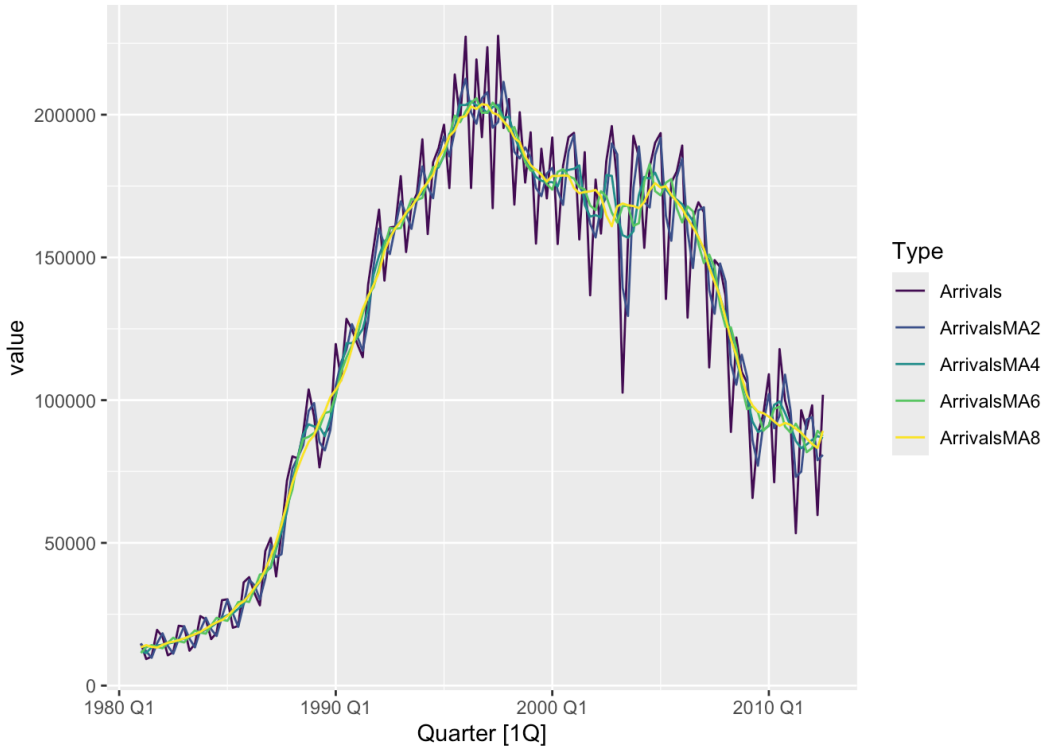
```
library(slider)
library(viridis)
```

```
Loading required package: viridisLite
```

```
aus_jap_arrivals <- aus_arrivals |>
  filter(Origin == "Japan") |>
  mutate(ArrivalsMA2 = slide_dbl(Arrivals, mean, .before = 1, .a
         ArrivalsMA4 = slide_dbl(Arrivals, mean, .before = 2, .a
         ArrivalsMA6 = slide_dbl(Arrivals, mean, .before = 3, .a
         ArrivalsMA8 = slide_dbl(Arrivals, mean, .before = 4, .a
```

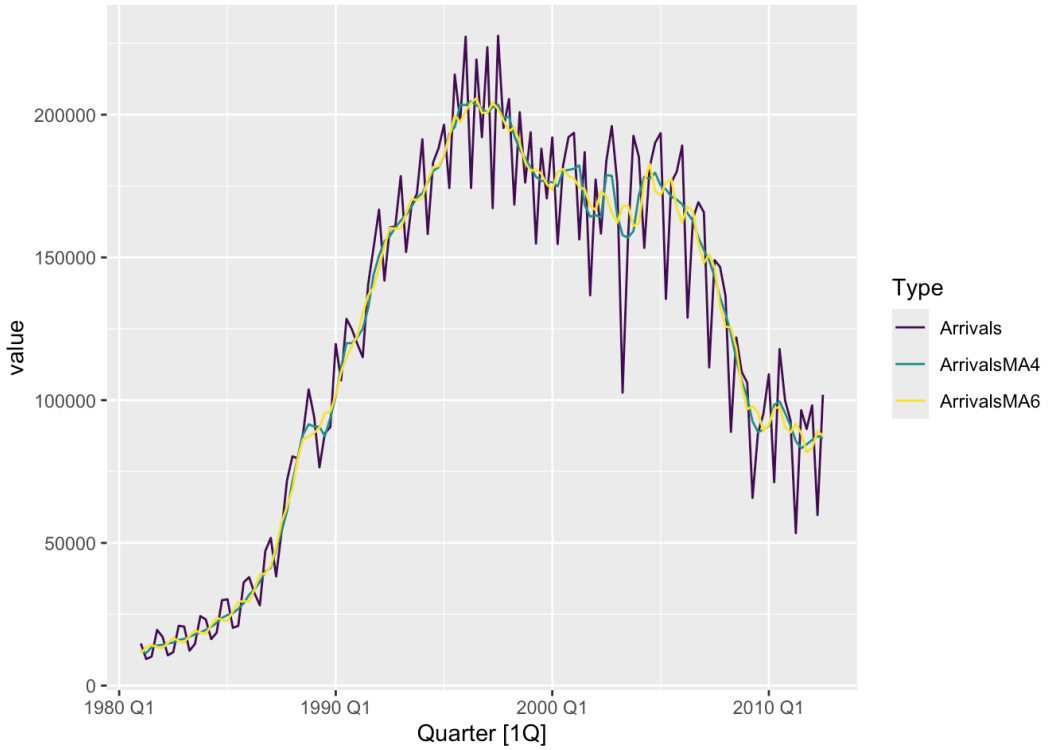
We can plot them individually, or plot all versions on the same plot.

```
aus_jap_arrivals |> select(-Origin) |>
  pivot_longer(cols = contains("Arrivals"), names_to = "Type")
  autoplot(value) + scale_color_viridis(discrete = TRUE)
```



There's a bit of overplotting here, but we generally see that the time series becomes smoother as we increase the window size. This is similar to bandwidth selection for kernel smoothing. On the other hand, MA4 seems to be smoother than MA6 (see the figure below). This is because the seasonality has period 4, which is averaged out when the window size of the moving average is a multiple of 4, but not otherwise.

```
aus_jap_arrivals |> select(-Origin) |>
  pivot_longer(cols = all_of(c("Arrivals", "ArrivalsMA4", "Arri
               names_to = "Type") |>
  autoplot(value) + scale_color_viridis(discrete = TRUE)
```

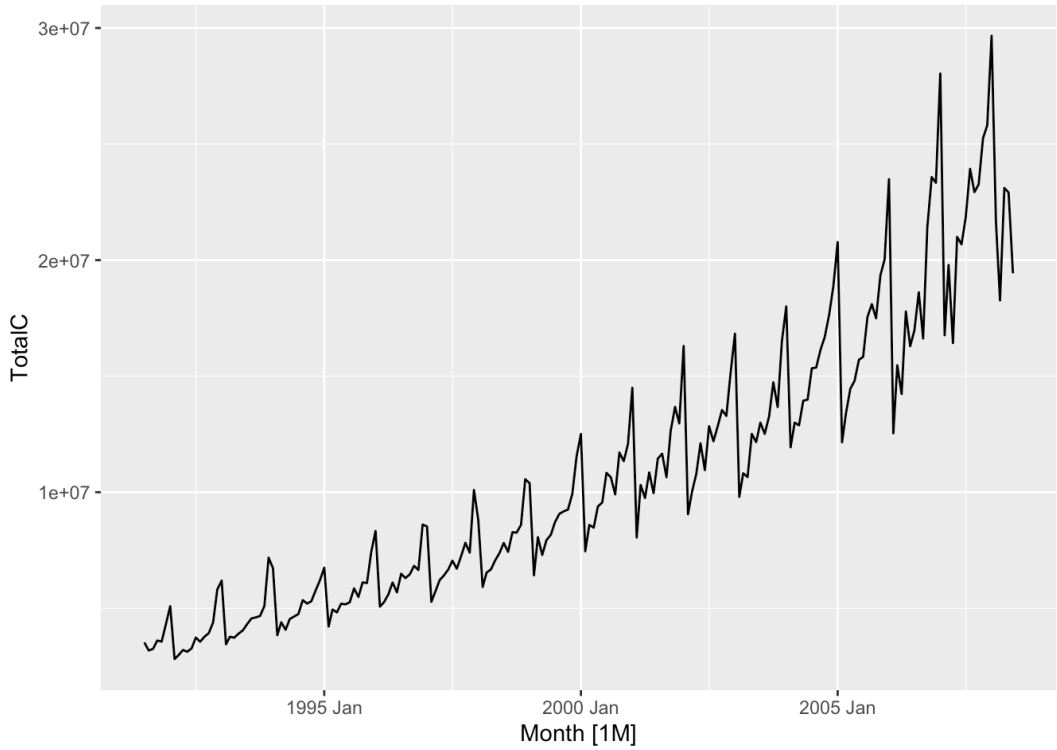


3. Log transform and differencing

Let's consider the sales of diabetes drugs in Australia.

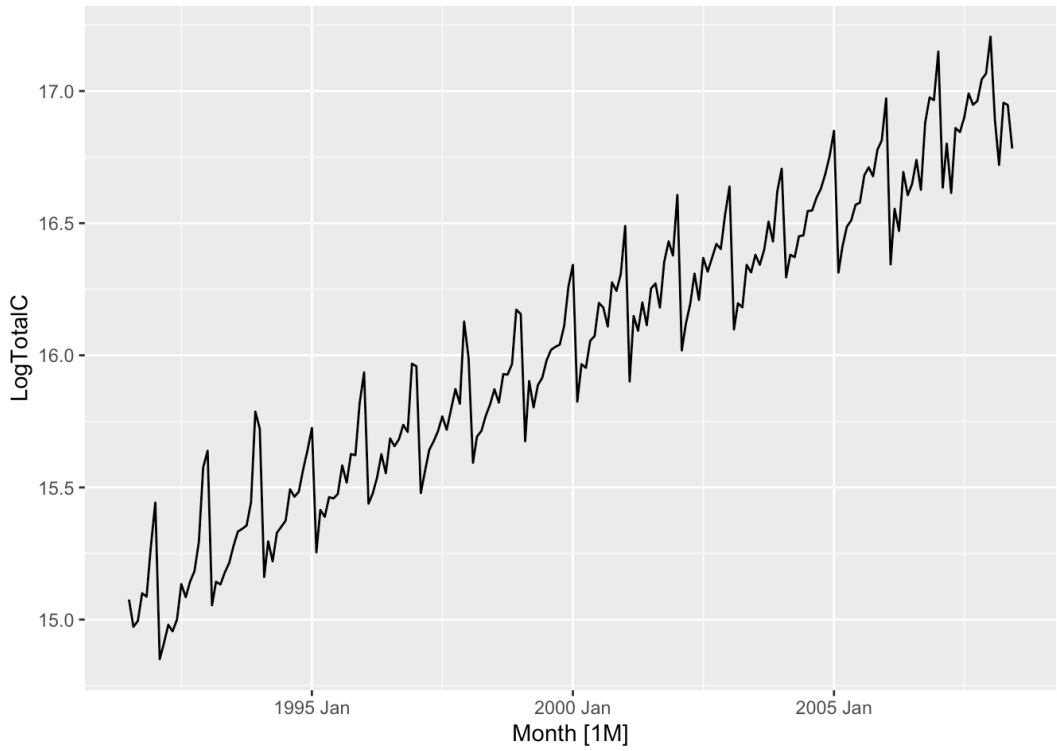
```
diabetes <- read_rds("../_data/cleaned/diabetes.rds")
diabetes |> autoplot()
```

```
Plot variable not specified, automatically selected `.vars =
TotalC`
```



The seasonal fluctuations grow with the level of the time series, so we shall apply a log transformation.

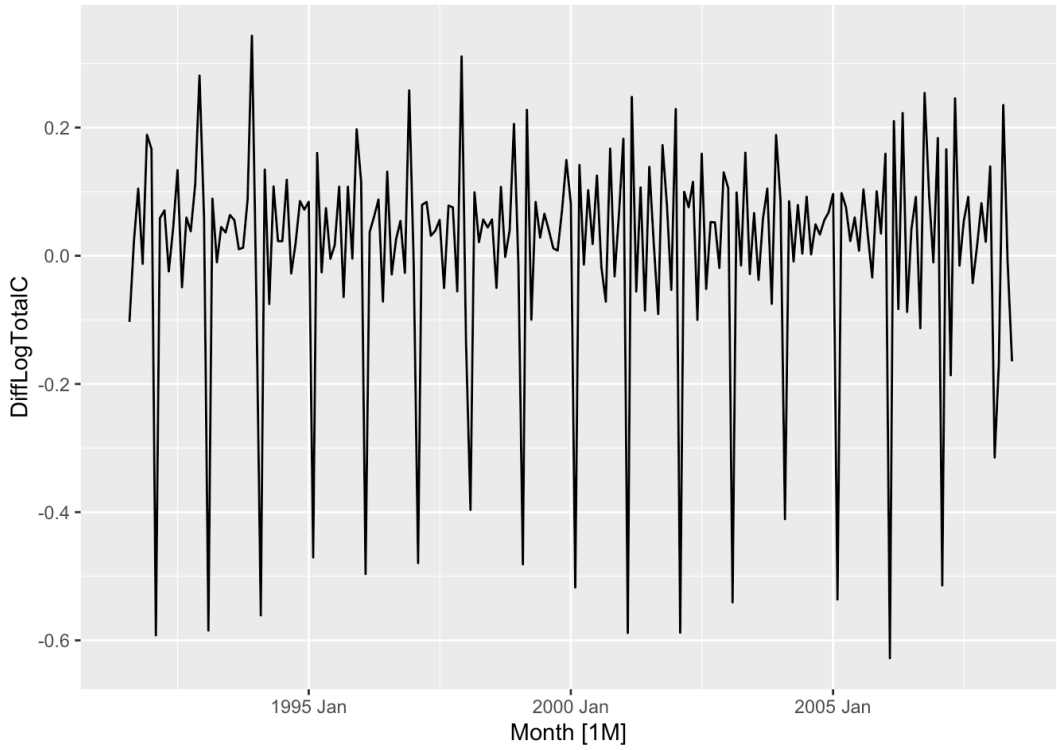
```
diabetes |>
  mutate(LogTotalC = log(TotalC)) |>
  autoplot(LogTotalC)
```



This is much better now. Let us now take differences

```
diabetes |>
  mutate(LogTotalC = log(TotalC)) |>
  mutate(DiffLogTotalC = difference(LogTotalC)) |>
  autoplot(DiffLogTotalC)
```

```
Warning: Removed 1 row containing missing values or values
outside the scale range
(`geom_line()`).
```

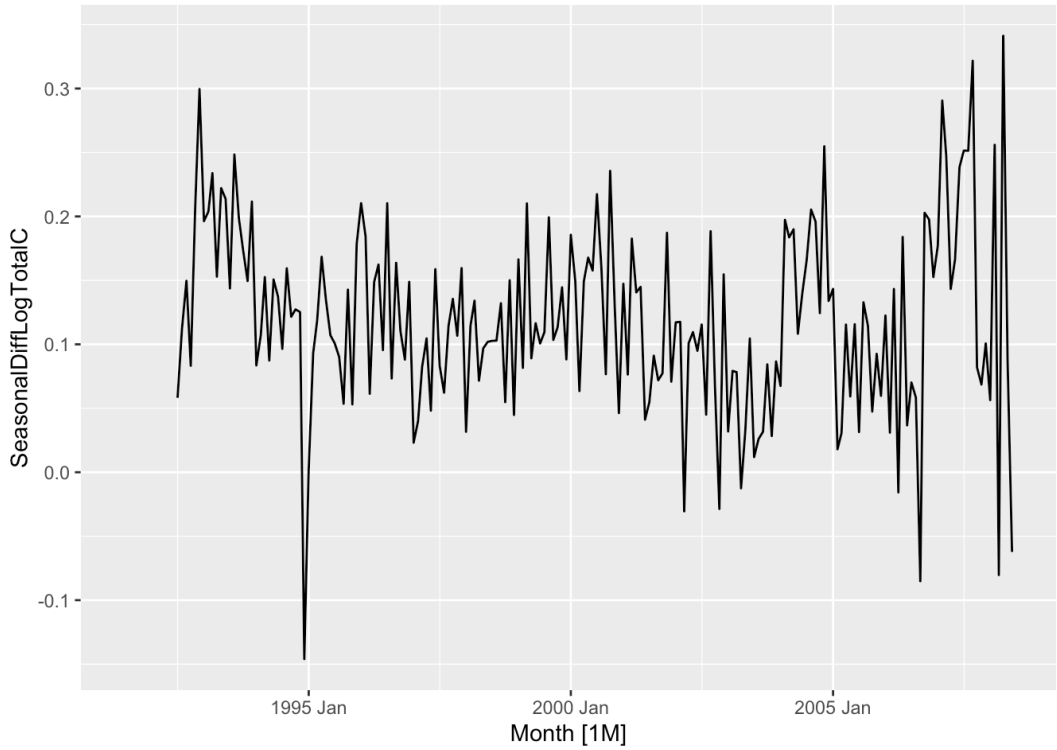


This measures the percentage *month-on-month* increase in sales of diabetes drugs. The time series still seems to be quite seasonal.

What happens if we take a seasonal difference?

```
diabetes |>
  mutate(LogTotalC = log(TotalC)) |>
  mutate(SeasonalDiffLogTotalC = difference(LogTotalC, 12)) |>
  autoplot(SeasonalDiffLogTotalC)
```

```
Warning: Removed 12 rows containing missing values or values
outside the scale range
(`geom_line()`).
```



It measures the *year-on-year* percentage increase in sales of diabetes drugs. The seasonality has disappeared! This series is also much more stable, see the y-axis limits.

4. Computing classical decomposition

Compute a classical decomposition for the following time series without using the `classical_decomposition()` *function.*

```
aus_arrivals_jap <- aus_arrivals |>
  filter(Origin == "Japan") |>
  select(Quarter, Arrivals)
```

We first create a trend and detrended columns.

```
detrended <- aus_arrivals_jap |>
  mutate(ArrivalsTrend = slide_dbl(Arrivals, mean, .before = 4,
         ArrivalsDetrended = Arrivals - ArrivalsTrend)
```

Add dummies for the quarters.

```
temp <- detrended |>
  mutate(Q1 = quarter(Quarter) == 1,
         Q2 = quarter(Quarter) == 2,
```

```
         Q3 = quarter(Quarter) == 3)
```

Compute seasonal and remainder components.

```
hard_decomp <- detrended |>
  mutate(ArrivalsSeasonal = lm(ArrivalsDetrended ~ Q1 + Q2 + Q3
         ArrivalsRemainder = ArrivalsDetrended - ArrivalsSeasona
```

Let us now run the automatic classical decomposition.

```
auto_decomp <- aus_arrivals_jap |>
  model(classical_decomposition(Arrivals)) |>
  components()
```

Comparing the two...

```
library(gridExtra)
```

```
Attaching package: 'gridExtra'

The following object is masked from 'package:dplyr':

    combine
```

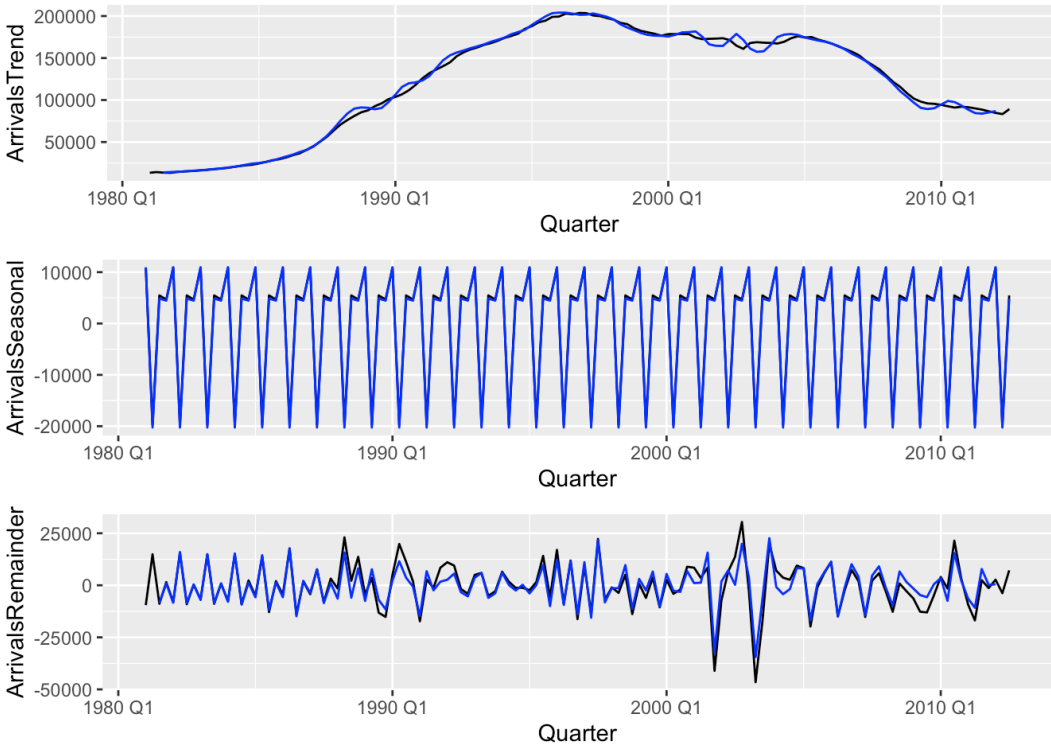
```
plt1 <- hard_decomp |>
  ggplot(aes(x = Quarter, y = ArrivalsTrend)) +
  geom_line() +
  geom_line(data = auto_decomp,
            mapping = aes(y = trend), color = "blue")

plt2 <- hard_decomp |>
  ggplot(aes(x = Quarter, y = ArrivalsSeasonal)) +
  geom_line() +
  geom_line(data = auto_decomp,
            mapping = aes(y = seasonal), color = "blue")

plt3 <- hard_decomp |>
  ggplot(aes(x = Quarter, y = ArrivalsRemainder)) +
  geom_line() +
  geom_line(data = auto_decomp,
            mapping = aes(y = random), color = "blue")

grid.arrange(plt1, plt2, plt3, nrow = 3)
```

```
Warning: Removed 4 rows containing missing values or values
outside the scale range
(`geom_line()`).
Removed 4 rows containing missing values or values outside the
scale range
(`geom_line()`).
```

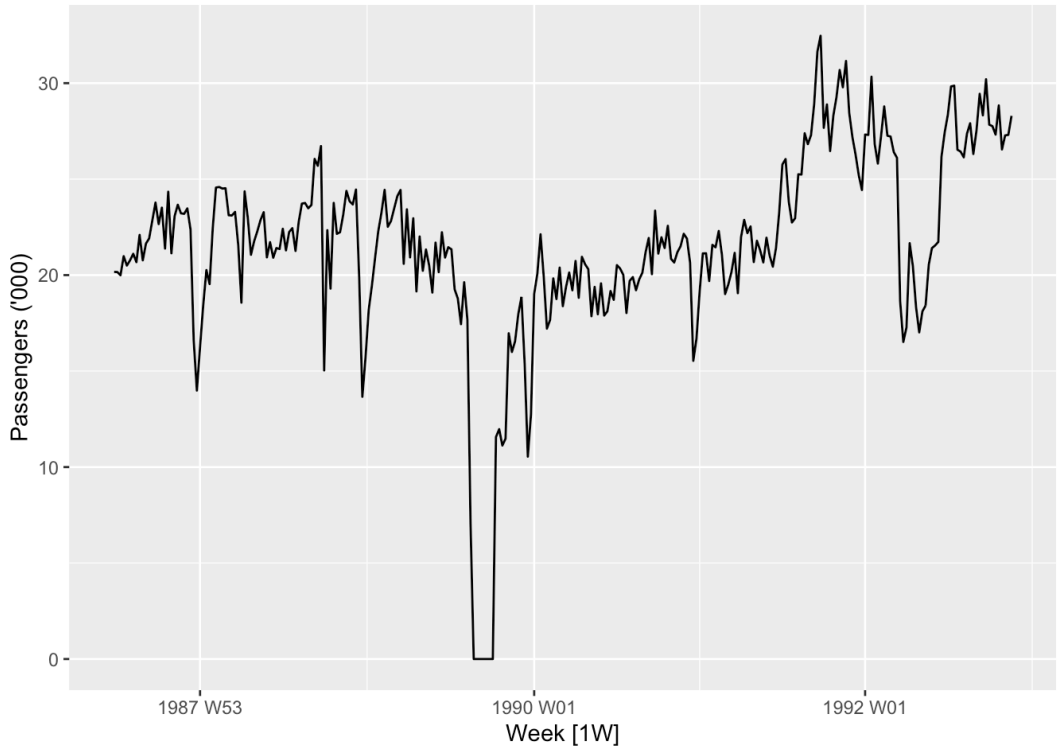


Quite similar. Some minor differences probably because of different window size for trend.

5. Classical vs STL decomposition

Start with the following code snippet creating a time series of passengers flying on Ansett Airlines. Perform classical and STL decompositions and comment on their differences. Which do you trust more?

```r
melsyd_economy <- ansett |>
  filter(Airports == "MEL-SYD", Class == "Economy") |>
  mutate(Passengers = Passengers/1000)
autoplot(melsyd_economy, Passengers) +
  labs(y = "Passengers ('000)")
```



If we try to compute a decomposition directly, we get an error saying that there are missing values. Let's first find the missing value and then fill it.

```
melsyd_economy |>
  scan_gaps()
```

```
# A tsibble: 1 x 3 [1W]
# Key:       Airports, Class [1]
  Airports Class       Week
  <chr>    <chr>     <week>
1 MEL-SYD  Economy 1987 W38
```

```
# Missing entry is 1987 W38
```

```
melsyd_economy |>
  filter_index("1987 W35" ~ "1987 W40")
```

```
# A tsibble: 5 x 4 [1W]
# Key:       Airports, Class [1]
      Week Airports Class   Passengers
    <week> <chr>    <chr>        <dbl>
1 1987 W35 MEL-SYD  Economy       20.8
2 1987 W36 MEL-SYD  Economy       21.6
3 1987 W37 MEL-SYD  Economy       21.9
```

```
4 1987 W39 MEL-SYD  Economy       23.8
5 1987 W40 MEL-SYD  Economy       22.7
```

Considering the nearby values, it seems reasonable to impute the mean of the preceding and succeeding weeks.

```
melsyd_economy <- melsyd_economy |>
  fill_gaps(Passengers = (21.9 + 23.8) / 2)
```

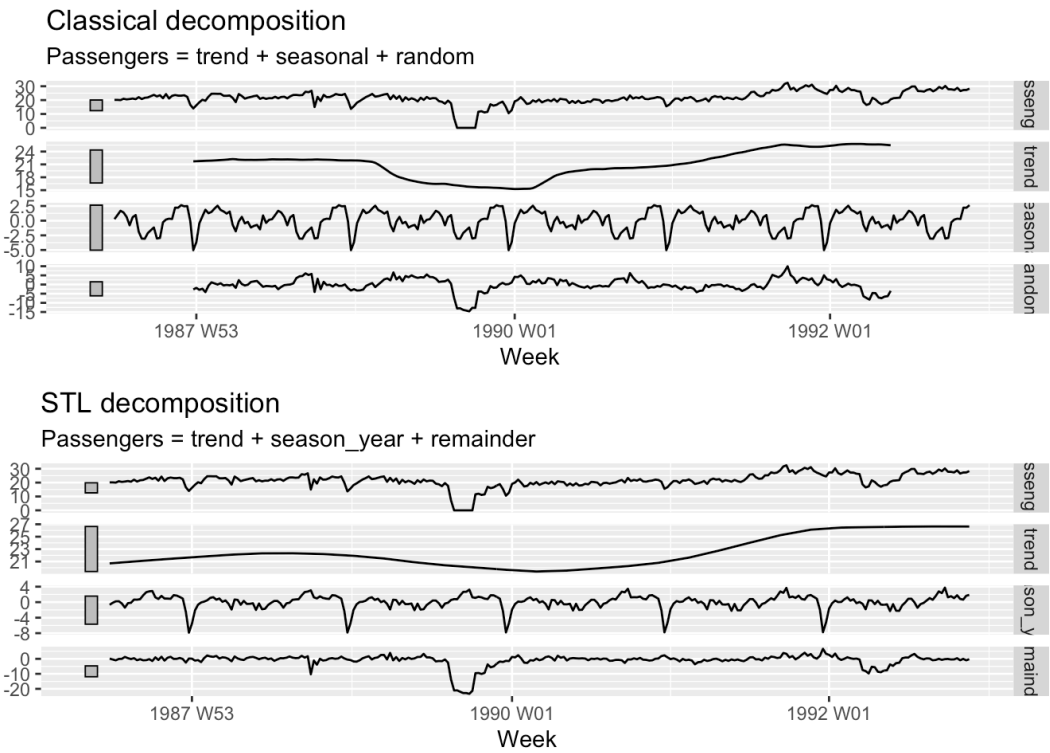
We now perform classical and STL decompositions.

```
plt1 <- melsyd_economy |>
  model(classical_decomposition(Passengers)) |>
  components() |>
  autoplot()

plt2 <- melsyd_economy |>
  model(STL(Passengers, robust = TRUE)) |>
  components() |>
  autoplot()

grid.arrange(plt1, plt2, nrow = 2)
```

```
Warning: Removed 104 rows containing missing values or values
outside the scale range
(`geom_line()`).
```



Here, we see that for classical decomposition, the zero passenger numbers between W34 and W40 in 1989 have caused a sharp downward shift over W34 to W40 in the seasonal component and also a significant dip in the trend component. This is undesirable because we know that these numbers are outliers—they are the result of an event that will not be repeated again (or at least that cannot be forecasted the available data). In comparison, the STL decomposition, after setting the option `robust = TRUE`, is able to put this portion of the time series entirely into the remainder component.

Furthermore, note that the classical decomposition is not able to estimate the trend component at the very start and end of the time series, while STL is able to do so.